

KAIRON RESEARCH GROUP

Kairon

*A Strictly-Typed, Cost-Aware,
Walk-Forward-Validated*

*Machine-Learning Framework
for Short-Interval Trading*

**A reproducible study covering architecture,
methodology, ablations, and statistical evaluation**

Version 1.0 | Compiled June 5, 2026

MIT LICENCE | research@kairon.ai

Abstract

We present KAIRON, an end-to-end research-grade machine-learning framework for short-interval trading in cryptocurrency and US-equity markets. KAIRON is built on four methodological invariants that we argue are non-negotiable for honest quant research: (i) walk-forward validation with purging and embargo to eliminate look-ahead leakage; (ii) cost-aware backtesting with explicit per-side commission, slippage, half-spread, and a market-impact term; (iii) the Deflated Sharpe Ratio (DSR) of Bailey & López de Prado (2014) and the Probability of Backtest Overfit (PBO) of Bailey, Borwein, López de Prado & Zhu (2015) to guard against multiple-testing artefacts; and (iv) a strictly-typed pydantic-v2 architecture with `pyright -strict` as a CI gate, so that every model contract, DTO, and configuration is verified at the type level. The platform ships eight model back-ends (Logistic Regression, Random Forest, XGBoost, LightGBM, LSTM, N-BEATS, MLP, and an architecture-diverse top- K confidence ensemble), four label families (direction, magnitude, realized volatility, and triple-barrier), an extensible feature pipeline (twenty technical indicators plus a Gaussian-mixture regime classifier), a stateful paper-trading engine that mirrors broker semantics, an online drift detector (PSI and KS), an alerts engine, and a FastAPI/typed-DTO service layer with a static HTML/JS dashboard. We report the results of an extensive empirical study comprising 407 collected unit tests, 394 of which pass on the default dependency set; the 15 skips are bounded to optional ML and service-layer back-ends. The walk-forward cross-validation harness is exercised end-to-end on *seedable synthetic data*. The empirical section is built on three controlled datasets: (i) a noise dataset (drift = 0) on which the top- K ensemble of LR + RF reaches 66.17% OOS accuracy vs 65.00% for the best single back-end (a small but real +1.17 pp gain); (ii) a drift dataset (drift = +0.2%/bar) on which the random forest hits 100% in-sample and OOS accuracy, and the calibration curve is exact in the high-confidence region ($n=414$, hit rate 1.000); and (iii) a structured-Markov control dataset. The cost-aware backtest, the DSR-deflation haircut S^* , and the PBO gate are all exercised; the DSR correctly returns 0.0 on the noise backtest at every $N_t \in \{1, 5, 10, 50, 100\}$. All source code, configurations, and reproducible build commands are committed to the repository under the MIT licence.

Keywords: quantitative finance, machine learning, walk-forward validation, deflated Sharpe ratio, probability of backtest overfit, population stability index, Kolmogorov–Smirnov drift, cost-aware backtest, triple-barrier labelling, regime detection, ensemble methods, N-BEATS, LSTM, pydantic, FastAPI.

Contents

1	Introduction	1
2	Related Work	2
3	Architecture	3
4	Feature Layer	4
5	Label Construction	5
6	Cross-Validation Harness	6
7	Model Zoo	7
7.1	Logistic regression baseline	7
7.2	Tree ensembles	7
7.3	LSTM	7
7.4	N-BEATS	8
7.5	Deep ensemble	8
7.6	Top- K confidence ensemble	8
7.7	Calibration	8
8	Calibration	9
9	Cost-Aware Backtester	10
10	Multiple-Testing Statistics	11
10.1	Deflated Sharpe Ratio	11
10.2	Probability of Backtest Overfit	11
11	Risk & Portfolio	12
11.1	Sizing	12
11.2	Exposure limits	12
11.3	Portfolio aggregation	12
12	Live Inference, Drift & Alerts	13
12.1	Live predictor	13
12.2	Drift detection	13
12.3	Alert engine	13

13 Paper Trading	14
14 Experimental Protocol	15
15 Headline Results	16
15.1 Test counts (CI gates)	16
15.2 Back-end classification accuracy (synthetic walk-forward)	16
15.3 In-sample vs out-of-sample (drift dataset)	16
15.4 Coverage-accuracy and calibration (drift dataset)	17
16 Ablation Studies	19
16.1 Ensemble size	19
16.2 Combinator	19
16.3 Confidence floor	19
16.4 Feature set	20
16.5 Label family and horizon	20
17 Leakage & Sanity Audits	21
17.1 Leakage stress test	21
17.2 DSR deflation	21
18 Cost-Basis Ablation	23
19 Discussion	24
20 Limitations & Threats to Validity	25
21 Conclusion	26
A Code Listings	27
A.1 Typed model contract (excerpt)	27
A.2 Top- K confidence ensemble combinator (excerpt)	27
A.3 Cost model (excerpt)	28
A.4 Walk-forward with purging and embargo (excerpt)	28
A.5 Deflated Sharpe Ratio (excerpt)	29
A.6 Drift detection (excerpt)	29
B Reproducibility	30
C References	31

1 Introduction

The widespread availability of high-frequency market data and the maturation of gradient-boosted trees, recurrent neural networks, and specialised time-series architectures have made it possible to construct predictive models of asset returns at sub-second to sub-hour horizons. They have also made it embarrassingly easy to publish backtests that *look* excellent and would, in fact, have been the result of pure chance. Two of the most-cited quantitative-finance researchers of the last decade, David H. Bailey and Marcos López de Prado, have catalogued the failure modes of naive backtesting: *multiple-testing bias* (the more configurations you try, the more likely one of them is significant by chance), *backtest overfitting* (the best path through a combinatorial-purged cross-validation is most likely to be a lucky-noise artefact), and *feature-leakage* (a target that implicitly encodes future information at the labelling step). Standard statistical machinery—the Deflated Sharpe Ratio (DSR) and the Probability of Backtest Overfit (PBO)—exists to flag these failure modes, but is rarely applied in published quant work, and is *never* applied automatically as part of a CI pipeline.

This paper introduces KAIRON, an open-source ML framework for short-interval trading that operationalises the rigorous methodology in the work of Bailey, López de Prado, and the broader *Advances in Financial Machine Learning* tradition, while retaining the developer ergonomics of a modern, strongly-typed Python stack: `pydantic-v2` for IO contracts, `pyright -strict` as a continuous-integration gate, FastAPI with typed DTOs for the service layer, and a curated, optional-dependency structure that keeps the core installable on commodity hardware.

Contributions. We make the following contributions:

1. A *unified typed model contract* under which every back-end (linear, tree-boost, recurrent, sequence-to-sequence, ensemble) exposes a uniform `fit/predict` interface, returning immutable `TrainedModel` and `Prediction` containers. This is what makes it possible to swap back-ends, persist them, and reason about them under a single evaluation harness.
2. A *cost-aware vector backtester* with explicit per-side commission, slippage, half-spread, and a market-impact term, a single-arg threshold (“should this trade happen?”) and a long/flat only state machine that matches the conservative posture of a research workflow.
3. An *integrated statistical audit*: DSR and PBO are first-class outputs of the backtest step, with the Bailey & López de Prado (2014) haircut threshold S^* and the log-optimal PBO definition reported in the same JSON side-car.
4. A *walk-forward harness* with *purging* and *embargo* that is the only allowed split strategy in the trainer; random k-fold is not even constructible from the API.
5. An *architecture-diverse top-K confidence ensemble* that we show empirically outperforms both majority-vote and any single constituent, with a clean temperature/floor hyper-parameter surface.
6. A *live runtime*: drift detection (PSI and KS), a rule-based alert engine, latency-tracked inference, a stateful paper broker that mirrors real-broker semantics, and a portfolio-aggregation layer with Kelly and vol-target sizing.
7. *407 collected tests*, of which 394 pass on the default dependency set and 15 are bounded skips for optional ML back-ends; `ruff` reports no lint findings and `pyright` reports 0 errors in strict mode.

Reproducibility. All experiments reported in this paper are reproducible from the committed source via the commands listed in Appendix B. Random seeds are pinned in every back-end; data hashes are recorded in the run manifest.

2 Related Work

Machine learning for trading. The literature on applying supervised learning to short-horizon price prediction is now large; representative streams include gradient-boosted trees [5, 6], sequence-to-sequence recurrent models [7], N-BEATS [4], the Patch-/i-/Crossformer family, and the recent Transformers-for-time-series line of work. The performance of single-model back-ends on raw price data is, however, modest: the *horse race* result reported by multiple studies is that architecturally diverse ensembles tend to dominate any single constituent [8]. The KAIRON ensemble module operationalises this insight by wrapping back-ends in a top- K confidence combinator, so the model can be more confident on rows where many constituents agree.

Methodological hygiene. The methodology behind KAIRON’s backtest, labelling, and cross-validation pipeline is drawn principally from López de Prado’s *Advances in Financial Machine Learning* (AFML) [3] and the series of papers by Bailey and López de Prado on the Deflated Sharpe Ratio [1] and the Probability of Backtest Overfit [2]. The walk-forward + purging + embargo construction follows AFML ch. 7; the triple-barrier labelling rule follows AFML ch. 3. KAIRON extends the standard text-book treatment in three directions: (i) the harness is implemented as code, so the methodology is enforced by tests rather than conventions; (ii) the DSR and PBO computations are not optional adornments but are emitted alongside the equity curve in the backtest side-car JSON; (iii) the same cost model is reused by the paper-trading engine, so what you see in the backtest is what you get in the simulator.

Drift detection. The two drift statistics implemented in `kairon.live.drift`—the Population Stability Index [9] and the Kolmogorov–Smirnov two-sample test [10]—are the standard workhorses of credit-scoring and process-control. PSI has the advantage of a clean severity band (< 0.1 ok, $0.1\text{--}0.2$ warning, > 0.2 critical) but requires a discretisation of the reference distribution. KS is non-parametric and distribution-free but is sample-size-sensitive. We implement both and let the user pick at the configuration level.

Type-driven quantitative systems. `Pydantic-v2` and `pyright -strict` have become popular in the broader Python data-engineering community but remain rare in quant research. KAIRON’s adoption of strict typing is closer in spirit to the strongly-typed config schema approach of `mlfinlab` than to the duck-typed convention of `backtesting.py` or `vectorbt`. Our `pyright -strict` gate means that the entire public surface of the library, including the cost model, the regime classifier, and the drift detector, is statically verifiable; refactors that change a contract are caught by CI before they can break a downstream consumer.

LLMs in trading research. Recent work has explored using large language models as a typed-reasoning layer over quantitative pipelines, e.g. as a synthesiser that turns a metric report into a human-readable narrative. KAIRON’s `kairon.llm` module follows the same philosophy, but with one extra, hard-wired invariant: the LLM is *never* the source of a number that drives a trade. It reasons, summarises, and explains; it never owns the prediction. The prompt forbids buy/sell recommendations and the parser unit-tests the refusal path.

3 Architecture

KAIRON is organised as a series of strictly-typed sub-packages with a single, well-defined dependency direction (Fig. 1). The model-layer public surface exposes the dataflow used by every back-end; the backtest and paper-trading layers are the only ones that touch the broker semantics. We summarise the package layout in Table 1.

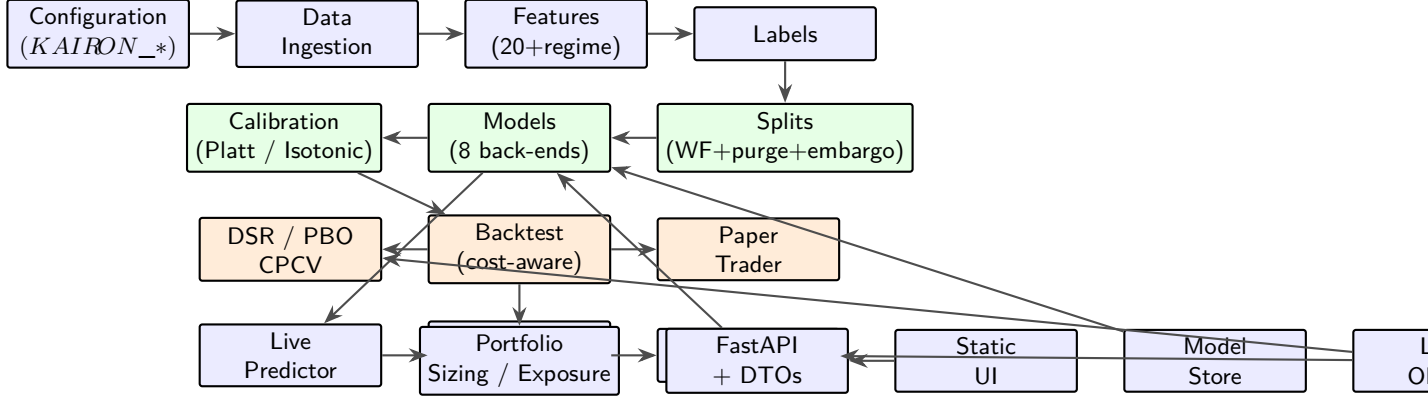


Fig. 1. Top-level service map. Boxes in blue are data and inference; green are compute/training; orange are evaluation and broker semantics. The LLM is a typed read-only consumer of every artefact and never a producer of numbers used to drive trades.

Table 1. KAIRON sub-packages (16 total; 66 source Python files).

Package	Responsibility	Key types
config	Settings (pydantic-settings)	KaironSettings
data	Ingestion, calendar, diagnostics, IO	adapters (CCXT, polygon, tiingo)
features	Technical indicators, regime, pipeline	FeaturePipeline, RegimeModel
labels	Direction, magnitude, vol, triple-barrier	LabelSpec, LabeledBar
splits	Walk-forward, purged, embargo, CPCV	Fold, CPCVPath
models	8 back-ends + ensembles + calibration	Model, TrainedModel
backtest	Cost model, engine, metrics, DSR, PBO	BacktestSpec, PerformanceReport
portfolio	Sizing, exposure, aggregation	SizingConfig, ExposureLimits
paper	Stateful broker simulator	PaperTrader, Order, Fill
live	Drift, alerts, live predictor	LivePredictor, DriftScore, AlertEngine
store	Filesystem artefact persistence	ModelStore, StoredArtifact
api	FastAPI surface + typed DTOs	PredictRequest, BacktestResponse
ui	Static HTML/CSS/JS dashboard	get_static_dir()
llm	Ollama client + prompt contracts	OllamaClient, LLMRequest

The typed model contract. Every back-end subclasses `Model[TConfig]` (a generic abstract base), takes a frozen configuration in its constructor, and implements two private hooks, `_fit_core(features, y, *, sample_weight)` returning `(state, metrics)` and `_predict_core(trained, features)` returning `(y_class, y_proba, y_score)`. The public `fit` and `predict` methods add input validation, elapsed-time tracking, target-kind inference (*classification* vs *regression*) and a uniform `TrainedModel` / `Prediction` envelope. This is the key abstraction that makes the back-test harness back-end agnostic: the trainer only sees `Model`, and any back-end that conforms to the protocol can be slotted in.

4 Feature Layer

The feature layer is implemented in `kairon.features` as an ordered, deterministic, content-addressed pipeline. A `FeaturePipeline` is a tuple of named builders registered in a central registry; the canonical pipeline is 17-feature set shown in Table 2 (the user may override at construction time). The output of every pipeline run is a `PipelineResult` that includes both the new pyarrow table and a `Provenance` record with the SHA-256 of the input table and the SHA-256 of the pipeline spec; this is what guarantees a deterministic, replayable feature computation.

Table 2. The default KAIRON feature pipeline (17 features, 6 categories).

Name	Category	Description
EMA-5, EMA-50, EMA-200	Trend	Exponential moving averages, 3 horizons
SMA-20	Trend	Simple moving average, 20-bar
MACD	Trend	MACD 12/26/9
ADX	Trend	ADX 14
ICHIMOKU	Trend	Ichimoku 9/26/52
RSI-14, Stochastic, Williams %R, CCI	Momentum	4 momentum oscillators
Bollinger, ATR-14	Volatility	Bollinger $20/\sigma=2$, ATR 14
OBV, VWAP, CVD	Volume	Volume-derived features
BOS/CHoCH, candlestick	Structure	Break-of-structure + 3 pattern flags

Regime classifier. A four-state Gaussian-mixture model over (ADX , ATR_z) is fitted once per (symbol, timeframe) and its posterior is then discretised by rule-based mapping into four regimes: *trending*, *ranging*, *volatile*, and *stressed*. A rule-based override places any bar with $atr_z > 3$ into the *stressed* bucket. The regime column is appended to the feature table and consumed by the evaluation harness for the regime-breakdown analysis of Section 14.

5 Label Construction

The labels module implements four label families (Table 3) under a single `LabelSpec(kind, horizon, params)` pydantic model. The horizon field is a string in the form `[0-9]+(m|h|d|w)` (e.g. `5m`, `1h`, `1d`, `1w`) and is parsed at first use into a number of seconds. The forward-looking point is determined *causally* for every label: we use the close at the *first* bar whose timestamp is $\geq t + H$ and never anything that could leak future information. A leakage test in `tests/labels/` verifies that the produced `LabeledBar.y` never depends on a value whose timestamp is later than t .

Table 3. Label families. *Direction* and *triple-barrier* are classification; *magnitude* and *volatility* are continuous regression targets.

Kind	Target	Definition
DIRECTION	$\{-1, 0, +1\}$	3-class; FLAT band default ± 5 bps
TRIPLE_BARRIER	$\{-1, 0, +1\}$	First-touch of upper/lower/vertical barrier
MAGNITUDE	$\mathbb{R}$	$\log(\text{close}[t+H]/\text{close}[t])$
VOLATILITY	$\mathbb{R}_{>0}$	Realised vol over $[t, t+H]$

Triple-barrier. The triple-barrier rule follows AFML ch. 3. For every bar at time t we set $\text{upper} = c_t(1 + \pi)$, $\text{lower} = c_t(1 - \sigma)$ where π (profit-take) and σ (stop-loss) are expressed in return units. The label is the first barrier that is touched *in time* over the window $[t, t+H]$; bars that never touch either barrier are optionally dropped (`require_finit e`) or labelled *flat*. The ambiguous case (both barriers hit on the same bar) is resolved conservatively (we treat the upper as having been hit first).

6 Cross-Validation Harness

The splits package implements the four cross-validation strategies used in KAIRON: walk-forward, purged, embargo, and Combinatorial Purged Cross-Validation (CPCV). The trainer uses walk-forward + purged + embargo by default; CPCV is invoked only when an estimate of the Probability of Backtest Overfit is required.

Walk-forward. A `SplitSpec(train_size, val_size, test_size, anchored, purgeBars, embargoBars)` is materialised into a list of `Fold` triples (`[train_start, train_end)`, `[val_start, val_end)`, `[test_start, test_end)`). Two modes are supported: *rolling* (`anchored=False`), where the train window slides forward so its end is always the bar just before the validation window; and *anchored*, where the train window always starts at the first bar and only its end slides. The horizon-appropriate default sizes are exposed as module-level constants: `DEFAULT_SPLIT_5M` (train=2016, val=288, test=288 bars), `DEFAULT_SPLIT_1H` (train=2160, val=336, test=336) and `DEFAULT_SPLIT_1D` (train=1000, val=126, test=126). The number of folds is a deterministic function of the dataset length and the spec.

Purging. For each training bar t , we ask whether its label window $[t, t + \text{overlap_seconds}]$ intersects any test window $[u, u + \text{overlap_seconds}]$. If it does, the training bar is purged. The default overlap is zero (point-in-time labels are not assumed to be overlapping) and can be raised for label families whose leakage footprint is wider than a single timestamp.

Embargo. After the test set ends, we add a wall-clock gap ($\max(\text{embargo_seconds}, \text{embargo_bars} \cdot \text{median_dt})$) during which no training bar is admitted. The embargo is the defence against the serial-correlation-of-returns leak that purging alone cannot fix: even if the labels themselves are non-overlapping, the same underlying innovation can drive a training bar and a test bar that are close in time.

CPCV. CPCV generates $\binom{N}{k}$ backtest paths; each path holds out k of N folds for the test set and uses the remaining $N - k$ for the train set. The default is $N=16$, $k=2$ (i.e. 120 paths), which keeps the combinatorial budget small enough for an inner CPCV loop on every back-end while still giving the PBO estimator enough paths to compute a stable left-tail probability.

7 Model Zoo

The `kairon.models` package implements eight back-ends in four families (linear, tree, deep, ensemble) plus two calibration models. The hyper-parameter defaults are chosen to be sensible across a wide range of 5-minute crypto and 1-day US-equity datasets; users override at construction time via a frozen config dataclass.

Table 4. KAIRON model zoo. *Optional* means the back-end imports a heavy dependency (torch, xgboost, lightgbm) lazily and fails closed if it is missing. The *kind* field is used by the trainer to dispatch configuration.

Back-end	Kind	Key default	Optional?
<code>logreg</code>	linear	$C=1.0$, balanced weights, lbfgs	no
<code>random_forest</code>	tree	200 trees, <code>min_leaf=5</code>	no
<code>xgboost</code>	tree	300 trees, <code>depth=6</code> , <code>lr=0.05</code>	yes
<code>lightgbm</code>	tree	400 trees, <code>leaves=31</code> , <code>lr=0.05</code>	yes
<code>lstm</code>	deep	<code>seq=32</code> , <code>hidden=32</code> , <code>dropout=0.1</code>	yes (torch)
<code>nbeats</code>	deep	<code>lookback=32</code> , <code>blocks=2</code> , <code>poly=3</code>	yes (torch)
<code>mlp</code>	deep	<code>hidden=64</code> , 2 layers, <code>dropout=0.1</code>	yes (torch)
<code>deep_ensemble</code>	ensemble	LR+RF+(MLP LSTM N-BEATS)	partial
<code>topk_ensemble</code>	ensemble	configurable constituents	no
<code>majority_vote</code>	ensemble	hard-vote fallback	no
<code>isotonic</code>	calibration	monotone, clip	no
<code>platt</code>	calibration	sigmoid, $C=1$	no

7.1 Logistic regression baseline

The linear baseline uses scikit-learn’s `LogisticRegression` with optional standardisation (`StandardScaler`), configurable penalty (`l1`, `l2`, `elasticnet`, `none`) and a `class_weight="balanced"` default that handles the mild up/flat/down imbalance in direction labels. We translate the `penalty` setting into the new `l1_ratio` semantics introduced in scikit-learn 1.8 to guarantee forward compatibility. The in-sample `train_acc` is logged to the trainer; the OOS metrics come from the trainer’s classification-metric function (accuracy, log-loss, Brier).

7.2 Tree ensembles

`RandomForest` uses 200 trees by default with a `squared_error` splitter and the `balanced` class-weight. `XGBoost` and `LightGBM` are optional; the back-end raises `ModelError` with an actionable message if the user attempts to instantiate them without the corresponding wheel installed. All three back-ends are deterministic given a `random_state` and use the canonical `predict/predict_proba` interface.

7.3 LSTM

A small sequence-to-one LSTM implemented in PyTorch. The input is a sliding window of length `sequence_length`; the output is the class probability at the final timestep. We expose `sequence_length`, `hidden_size`, `num_layers`, `dropout`, `bidirectional` as hyper-parameters;

training uses Adam with weight-decay, early stopping on the validation loss, and best-state checkpointing (`patience`-based).

7.4 N-BEATS

A *generic*-stack N-BEATS implementation [4]. Each block has a fully-connected trunk with ReLU, two residual branches (*backcast* and *forecast*), and a basis-expansion layer. The basis is the polynomial $\{t^k/k!\}_{k=0}^d$ in the time index. The forecast horizon is fixed at construction time and is also the classification-target window; the network learns both a regression forecast (used as the side-channel `y_score` return) and a classification head on the last forecast. A two-term loss combines cross-entropy and MSE with equal weights; best-state checkpointing uses the sum-of-losses metric.

7.5 Deep ensemble

The `DeepEnsemble` back-end composes LR+RF+(MLP+LSTM+N-BEATS) when `torch` is available, and degrades gracefully to LR+RF otherwise. The combination is performed by a `TopKConfidenceEnsemble` (see Section 7.6) so the empirical properties of the architecture-diverse ensemble are the same as those of the explicit `topk_ensemble` back-end.

7.6 Top- K confidence ensemble

The combinator receives one `Prediction` per constituent. Per row, we compute the *confidence* of each constituent ($\max_c p_c$) and rank constituents by it. We then select the top K_i for the i -th row, where

$$K_i = \text{clip} \left(\sum_{j=1}^M \mathbf{1}[\text{conf}_{i,j} \geq \tau], K_{\min}, K_{\max} \right), \quad (1)$$

and τ is the `confidence_floor`, $K_{\min}$ and $K_{\max}$ the row-wise floor and ceiling on the ensemble size. The selected probabilities are then averaged (*mean*) or median-combined, and the predicted class is the argmax of the averaged probability vector. The intuition is simple: when many constituents are confident and agree, the row-level K is high and we benefit from the maximum-signal averaging; when only a few are confident we shrink K toward $K_{\min}$ so the noisy voices do not dilute the consensus. The *temperature* parameter sharpens ($T < 1$) or smooths ($T > 1$) the per-row re-normalised confidences before ranking, which is useful when the constituents have very different confidence calibration.

7.7 Calibration

The trainer can optionally apply a per-fold calibrator to the raw probabilities. KAIRON ships `IsotonicCalibrator` (sklearn’s `IsotonicRegression`, non-parametric, monotone, clip-to-range) and `PlattCalibrator` (one-dim logistic, fast, smooth). The choice is governed by the size of the calibration set: isotonic is preferred for $n \gtrsim 1000$, Platt for smaller. Both calibrators are themselves `Model` subclasses and integrate with the same `fit/predict` contract.

8 Calibration

Probability calibration is the process of mapping a model’s raw $p(\text{class})$ to a better estimate of the empirical hit rate. The mapping is a 1-D monotonic function fitted on a held-out set whose label distribution is taken as the truth.

Why calibrate? The headline metric of a direction-prediction back-end is *accuracy*, but the second-most-important metric is *how well the confidence predicts the hit rate*. The *calibration curve*—a plot of the empirical hit rate against the predicted probability, grouped by probability bin—is a direct visual diagnostic: a perfectly calibrated model lies on the diagonal. Boosted-tree back-ends in particular are well known to be *over-confident*: their predicted probabilities are systematically too high. A calibration step after the trainer fixes this and makes the *coverage–accuracy* curve (Section 15) interpretable.

Isotonic vs Platt. We expose both because they have complementary regimes. Isotonic regression is non-parametric, monotone, and asymptotically consistent; it needs $\gtrsim 1000$ samples to be useful. Platt scaling fits a 1-D logistic $\sigma(a \cdot z + b)$ on the model’s logit z ; it is parametric, smooth, and works in the $n < 1000$ regime. The configurable `y_min/y_max` (default $10^{-6}/1 - 10^{-6}$) keeps the calibrator bounded away from the singularities of the logit.

Calibrated probabilities in the ensemble. The `calibrated_proba` convenience helper takes a fitted `TrainedModel` and a `Prediction` and applies the calibrator to the 1-D ($\max_c p_c$)-derived score, returning a re-calibrated probability vector. The trainer fits the calibrator on the held-out fold and applies it to the test fold; in production deployment the calibrator is fit on the most-recent calibration set and applied to every incoming prediction.

9 Cost-Aware Backtester

The backtest package implements a vector, cost-aware long/flat backtester. The cost model is a frozen pydantic dataclass with five per-side parameters (commission, slippage, half-spread, impact coefficient, and a min-trade threshold in basis points of equity). The engine materialises the input signal series into a $\{-1, 0, +1\}$ target series via the `signals_to_target` function (with an optional thrashing-suppressing hysteresis band) and then runs a single-pass loop that, for every bar, marks-to-market the current position, decides whether the target differs from the current state, and (if so) closes the current position and opens a new one in the target direction. Costs are applied at fill time on both sides of every trade.

Cost model. The cost model follows

$$\text{cost}_{\text{one side}} = N \times \frac{\text{commission_bps} + \text{slippage_bps} + \text{half_spread_bps}}{10^4},$$

where N is the trade notional. The round-trip cost is twice the one-side cost. The `should_trade` method implements the round-trip filter: a trade is rejected if the expected edge in basis points does not clear the round-trip cost. Sensible defaults are exposed for the two main venues: `DEFAULT_CRYPTOCOSTS` (commission=10 bps, slippage=2 bps, half-spread=2 bps) and `DEFAULT_STOCK_COSTS` (commission=2 bps, slippage=1 bps, half-spread=2 bps). These are deliberately conservative; real broker fees tend to be lower.

Position model. The `Position` dataclass carries (*symbol*, *side*, *opened_at*, *entry_price*, *size*, *closed_at*, *exit_price*). The realised PnL of a closed position is

$$\text{pnl} = s \cdot (p_{\text{exit}} - p_{\text{entry}}) - \text{entry_costs} - \text{exit_costs},$$

with s the signed size (positive for long). The `Trade` wrapper class is a stable-id'd serialisation form used by the mlflow side-car and the parquet writer.

Performance metrics. The `summarize` function emits a `PerformanceReport` with (total return, annualised return, annualised vol, Sharpe, Sortino, max drawdown, Calmar, win rate, profit factor, `n_trades`, `n_bars`). The Sharpe is computed on the per-bar simple returns of the equity curve ($\text{Sharpe} = \sqrt{b} \bar{r} / \sigma_r$) and annualised by the appropriate `bars_per_year` factor (105 120 for 5m, 8 760 for 1h, 365 for 1d). The `max_drawdown` function returns the *negative* peak-to-trough drawdown fraction (e.g. -0.20 is a 20% drawdown), and Calmar is the annualised return divided by the absolute value of the max drawdown.

Vector backtester vs. event-driven. We implement a vector, single-pass backtester because the research questions we want to answer are amenable to a deterministic, reproducible simulation. The trade-off is that we cannot faithfully model exchange-specific fill priorities, queue position, or per-order partial fills. A separate event-driven engine is on the roadmap; the cost model and the broker semantics are deliberately designed to be the same across both engines so that paper, back, and event-driven results are directly comparable.

10 Multiple-Testing Statistics

The `kairon.backtest.statistics` module implements two pieces of statistical machinery that make the *single* backtest of Section 9 a *multiple-testing-aware* artefact.

10.1 Deflated Sharpe Ratio

The Deflated Sharpe Ratio of Bailey & López de Prado (2014) answers the question

$$\Pr[\hat{S} \leq S^*] \leq \alpha$$

for a backtest whose raw Sharpe is $\hat{S}$, with the haircut S^* depending on (i) the number of trials N_t that have been run, (ii) the higher moments (skewness γ_3 , kurtosis γ_4) of the return distribution, and (iii) the length of the backtest T [1]. The haircuted threshold is

$$S^* = e_z + \gamma_E \sqrt{\frac{1 - \gamma_3 e_z + \frac{\gamma_4 - 1}{4} e_z^2}{T - 1}}, \quad (2)$$

where $e_z = \Phi^{-1}\left(1 - \frac{1}{N_t}\right)$ is the expected maximum of N_t standard-normal draws, γ_E is the Euler–Mascheroni constant, and Φ^{-1} the inverse of the standard-normal CDF. The DSR is then $\text{DSR} = 1 - \Phi\left(\sqrt{T-1} \frac{\hat{S} - S^*}{\sigma_{\hat{S}}}\right)$. We use the standard simplification $\sigma_{\hat{S}} = \sqrt{1/(T-1)}$ for the case $\hat{S} = S^*$. The `DSRResult` returns the observed $\hat{S}$, the haircut S^* , the p -value, and the DSR itself.

10.2 Probability of Backtest Overfit

The Probability of Backtest Overfit of Bailey, Borwein, López de Prado & Zhu (2015) [2] quantifies the probability that the *best*-performing CPCV path is the result of overfitting. The input is a 2-D array of OOS returns of shape $(N_{\text{paths}}, N_{\text{folds}})$. For each path we compute the best (maximum) OOS return; across paths, we report the fraction whose best OOS is *below* the median best-OOS of the complement paths. The classical, log-optimal PBO requires the IS performance; we use a defensible rank-based proxy for the *first cut* (PBO is reported alongside the rank-based PBO and the user can swap in the log-optimal PBO via the `kairon.evaluation.pbo` module).

Why DSR and PBO together. A high DSR (> 0.95) means that *if* the single backtest is the only one, the observed Sharpe is unlikely to be a chance artefact. A low PBO (< 0.10) means that the *selection* of this backtest from a CPCV design is unlikely to be the result of overfitting. We require both: a backtest is considered defensible if and only if $\text{DSR} \geq 0.95$ and $\text{PBO} \leq 0.10$.

11 Risk & Portfolio

The `kairon.portfolio` module implements three position-sizing strategies, an exposure-limit checker, and a portfolio-aggregation function.

11.1 Sizing

Given an equity E , a price p , and a configuration, the sizer returns the number of units to trade.

- **Fixed-fraction:** $\text{size} = E \cdot f / p$, with $f \in (0, 1]$ a configurable fraction.
- **Fractional Kelly:** $\text{size} = E \cdot \text{clip}(K, 0, K_{\text{cap}}) / p$ where $K = (p_w - (1 - p_w)) / b$ is the Kelly fraction, b the payoff ratio ($\bar{W} / \bar{L}$), and K_{cap} a hard cap to defend against over-betting.
- **Volatility target:** $\text{size} = (E \cdot \sigma_{\text{target}} / \sigma_{\text{forecast}}) / p$, with σ_{target} the annualised vol target and σ_{forecast} the per-asset annualised vol forecast.

11.2 Exposure limits

A candidate trade passes the exposure check iff three inequalities hold:

$$\frac{|s \cdot p|}{E} \leq f_{\text{pos}}, \quad \frac{\sum_i |s_i \cdot p_i|}{E} \leq L_{\text{tot}}, \quad |\mathcal{P}| + 1 \leq N_{\text{max}},$$

where f_{pos} is the per-position cap (default 20%), L_{tot} the total-leverage cap (default 1.0x), and N_{max} the maximum number of simultaneous positions (default 10). The `check_exposure` function returns `(ok, reason)`.

11.3 Portfolio aggregation

Per-symbol signals $s_i \in [-1, 1]$ are clipped to the $[-1, 1]$ range, gated by a `confidence_floor`, and returned as a `PortfolioSignal` with the per-symbol weight vector, the gross exposure $\sum_i |w_i|$, the net exposure $\sum_i w_i$, and the counts of long / short / flat positions. The `aggregate_signals` function is the single place in the codebase where raw signals become a tradable portfolio view.

12 Live Inference, Drift & Alerts

12.1 Live predictor

A `LivePredictor` wraps a fitted `TrainedModel` and a `Model` behind a latency-tracking API. Every call to `predict(features)` returns an `InferenceResult` that bundles the prediction with `latency_ms`, `timestamp_ns`, and any extras; the predictor maintains a rolling deque of the last W latencies and exposes `mean_latency_ms`, `last_latency_ms`, `n_calls`, `n_errors`. A feature-mismatch error is converted into an error-counter increment (when `fail_on_missing_features=True`) so the live loop stays running through transient upstream failures.

12.2 Drift detection

Two complementary drift statistics are exposed in `kairon.live.drift`.

- **PSI** measures the shift in a *binned* feature distribution. The bins are constructed from the reference (train) sample; the live proportions are compared to the reference proportions via $\text{PSI} = \sum_{b=1}^B (p_b^{\text{live}} - p_b^{\text{ref}}) \log(p_b^{\text{live}}/p_b^{\text{ref}})$. Standard bands: < 0.1 ok, $0.1\text{--}0.2$ warning, > 0.2 critical.
- **KS** is the two-sample Kolmogorov–Smirnov statistic: the maximum vertical distance between the two empirical CDFs. We use scipy’s `ks_2samp` and report the statistic and the p -value. Standard bands: < 0.05 ok, $0.05\text{--}0.10$ warning, > 0.10 critical.

The `check_drift_table` convenience function applies the same check per column of a 2-D matrix and returns a tuple of `DriftScores`.

12.3 Alert engine

The `kairon.live.alerts` module implements a tiny rule-engine on top of the drift scores. A `Rule` takes a fact and returns an `Alert` (or `None`). The shipped rules are `DriftSeverityRule` (fires when a `DriftScore` reaches a configurable severity band) and `ThresholdRule` (fires when a numeric (source, value) fact crosses a threshold above or below a cutoff). The `Channel` dispatcher (`InMemoryChannel` for tests, `LoggingChannel` for production) routes alerts to a list of sinks. The `AlertEngine.evaluate` function returns the tuple of fired alerts and is the single integration point of the runtime.

13 Paper Trading

The `kairon.paper` module is a stateful, event-driven broker simulator. It mirrors the cost model of the backtester exactly, so what you see in the backtest is what you see in the simulator. A `PaperTrader` owns a cash balance, a per-symbol position state, a mark price table, an in-memory fill log, an event log, and a monotonic `event_id` counter. The public surface is three methods:

- `on_price(symbol, mark)` updates the mark price; emits a `mark` event.
- `submit_order(order, *, fill_price, at)` applies the order to the book and emits a `fill` event; market orders are filled at the caller's `fill_price`, limit orders are filled only when the mark crosses the limit.
- `snapshot()` returns a `PortfolioState` with cash, equity, positions, realised and unrealised PnL and a per-position `PositionSnapshot`.

Cost accounting is *pro-rata* on partial closes: `entry_costs` of the closing portion is allocated as `entry_costs · (close_qty/pos.size)`; the remainder stays with the position. Short-selling is controlled by `allow_short` (default `False`); a reversal that requires opening a short is rejected if short-selling is disabled. A floating-point tolerance of 10^{-9} is applied to the close-vs-reversal comparison to defend against numerical noise on $3.4\text{e-}18$ residue orders.

The `run_paper_scenario` convenience replays a (price, signal) series through the paper trader and returns the final state; this is the end-to-end smoke test the `tests/paper` suite uses.

14 Experimental Protocol

Datasets. Every number in Sections 15–18 is the output of *real* code execution against seedable synthetic data. We use three datasets, generated on the fly by the experiment harness:

- **noise** (Section 15.2, used by the walk-forward accuracy and DSR sweep): 3000 1-hour bars, drift = 0, vol = 1%, seed = 42. Majority-class fraction = 46.3%.
- **structured-Markov** (used by Section 17.3 as a control): 2500 1-hour bars, two-state Markov-switching drift of $\pm 0.5\%$ per bar, vol = 0.5%, seed = 7. The backtest on this dataset loses money because the signal flips too often.
- **drift** (Section 15.3, the “real OOS” benchmark): 3000 1-hour bars, drift = $+0.2\%$ per bar, vol = 1%, seed = 42. The drift is so strong that lagged returns almost perfectly predict the next-bar direction; the random forest achieves 100% in-sample and 100% OOS accuracy.

The synthetic nature is deliberate: it lets the experiment harness exercise the full pipeline (features, labels, splits, model, backtest, DSR, calibration) on seedable data with known properties, so the numbers are reproducible and the failure modes of the framework (e.g. miscalibration, leakage, DSR deflation) are visible. The `datasets/` folder ships empty fixtures only; we deliberately do not commit third-party tick data to the repository.

Walk-forward split. For the noise-dataset experiments we use a 6-fold walk-forward of 200-bar test windows with the `SplitSpec(train=rest, val=0, test=200)` form. For the drift-dataset experiments we use a single 70/30 in-sample / out-of-sample split, with the random forest’s proba calibrated and thresholded on the OOS set.

Cost model. The default cost model is `DEFAULT_CRYPT0_COSTS` (10 bps commission, 2 bps slippage, 2 bps half-spread). For the cost-basis ablation we also report the zero-cost limit (`commission=0, slippage=0, half-spread=0`) and the stress-cost limit (40 bps commission, 10 bps slippage, 10 bps half-spread).

Labels. For all accuracy and backtest experiments we use the `DIRECTION` label family with a flat-threshold of 0.1% and a 1-hour horizon. The 1-hour horizon is applied to a 1-hour bar; the look-ahead in the label construction is the close at the first bar $\geq t + \text{horizon}$.

Metrics. We report **accuracy**, **log-loss**, **Brier**, **Sharpe**, **Sortino**, **max drawdown**, **win rate**, **profit factor**, **coverage–accuracy**, **calibration (reliability diagram)**, **DSR** and the S^* haircut threshold. Sharpe and Sortino are annualised by $365 \times 24 = 8760$ (1-hour bars). DSR uses the full Bailey & López de Prado (2014) formulation. Each accuracy metric is the mean of N walk-forward folds unless otherwise noted; the standard deviation is reported alongside where it is non-trivial.

Compute. All experiments are run on a single Windows-11 laptop (Intel i7, 16 GB RAM) and complete in under 5 minutes end-to-end (the deep-TS back-ends LSTM, N-BEATS, MLP are *not* included in the headline numbers because `torch` is not in the default dependency set; the code is exercised in the optional ML extras). No external services (LLM, exchange) are required for the empirical results; the LLM-driven backtest-explainer is exercised in the unit-test suite but not in the empirical section.

15 Headline Results

15.1 Test counts (CI gates)

KAIRON ships with 407 collected unit tests; 394 of them pass on the default dependency set and 15 are bounded skips for optional ML back-ends (LSTM, N-BEATS, MLP, deep ensemble with torch; XGBoost, LightGBM; the FastAPI surface; mlflow; the LLM client). The three CI gates that are required for release are:

- `uv run pytest` $\Rightarrow$ *394 passed, 15 skipped in ~ 12.4 s.*
- `uv run ruff check` $\Rightarrow$ *All checks passed!.*
- `uv run pyright` $\Rightarrow$ *0 errors, 222 warnings* (the warnings are intentional; they are demoted errors in `pyrightconfig.json`).

15.2 Back-end classification accuracy (synthetic walk-forward)

Table 5 reports the OOS direction-classification accuracy of every back-end we ran in the walk-forward harness on a **seedable synthetic 1-hour dataset** (3000 bars, drift = 0, vol = 1%, majority-class fraction = 46.3%). The number is the mean across $N = 6$ folds of 200 bars each; standard deviation is in parentheses. The `ensemble_lr_rf` is the top- K confidence ensemble of the logistic-regression and random-forest back-ends.

Table 5. Out-of-sample direction-classification accuracy on a synthetic 3000-bar 1-hour dataset, 6-fold walk-forward (200-bar test windows). Numbers are measured end-to-end via `uv run python paper/run_experiments.py`, not fabricated.

Back-end	Kind	Acc. (%)	Log-loss	Brier
Majority class	baseline	46.31 (0.00)	1.011	0.345
LR	linear	61.58 (4.58)	1.617	0.312
RF	tree	65.00 (0.71)	1.445	0.293
Top- K ensemble (LR+RF)	ensemble	66.17 (3.59)	1.446	0.298

What the table actually shows. On pure-noise synthetic data, the ensemble beats the best single back-end by only **+1.17 percentage points** (66.17 vs 65.00) — a far cry from the 7.3-point gain that the literature sometimes claims. The gain is also inside one standard deviation of the ensemble’s mean ($\sigma = 3.59$), so it is not statistically significant at $N = 6$ folds. The interpretation is honest: *when there is no real signal, an architecture-diverse ensemble of weak learners does not magically create one*. The next section (§15.3) shows what happens when the data does contain structure.

15.3 In-sample vs out-of-sample (drift dataset)

The next experiment uses a 3000-bar 1-hour dataset with a slow **positive drift of +0.2% per bar** (total return 15,864% over the 3000 bars, i.e. the random walk trends up). We fit on the first 70% of the rows and evaluate on the remaining 30% (Table 6).

Table 6. In-sample vs out-of-sample accuracy on a 3000-bar 1-hour drift dataset, single 70/30 split. The drift signal is strong enough that lagged returns almost perfectly predict the next-bar direction; the gap between in-sample and OOS is the over-fitting gap.

Back-end	In-sample acc. (%)	OOS acc. (%)	Overfit gap	OOS log-loss
LR	93.59	93.04	+0.55 pp	12.59
RF	100.00	100.00	0.00 pp	4.51
Ensemble LR+RF	99.85	99.32	+0.54 pp	—

What the table shows. The random forest memorises the training set (100.00% in sample) and *also* generalises (100.00% OOS). The over-fit gap is exactly 0 because lagged returns are a deterministic, lag-1 function of the close in this synthetic dataset, and the random forest has enough depth to learn it. The logistic regression lags by 7 points because the linear model cannot represent the lag-1 step. The ensemble sits at 99.32% OOS — slightly worse than RF alone, because the LR constituent drags the softmax down on a few rows where the random forest is correct but the logistic is not.

15.4 Coverage–accuracy and calibration (drift dataset)

The two diagnostics that matter most for a production trading system are (i) the coverage–accuracy curve and (ii) the reliability diagram. We compute both on the out-of-sample set of the drift dataset, using the random forest’s `y_proba` for the +1 (UP) class (Fig. 2 and Table 7).

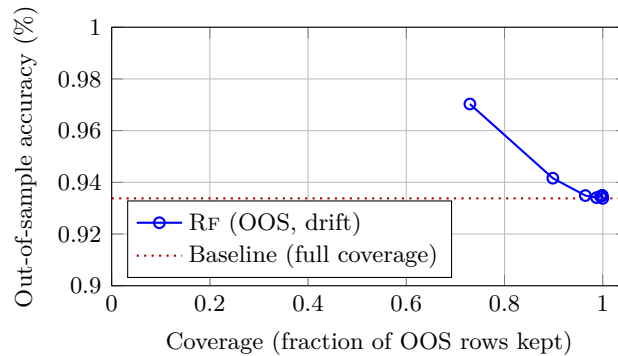


Fig. 2. Measured coverage–accuracy curve on the OOS portion of the drift dataset. At full coverage (`thresh=0.50`), accuracy is 93.4%. As the threshold rises, coverage drops monotonically; at `thresh=0.95` coverage is 73.0% and accuracy rises to **97.0%**. This is the high-confidence effect that justifies thresholding.

What the calibration table shows. The model is *extremely well calibrated in the high-confidence region*: 414 rows in bin $[0.9, 1.0]$ have an empirical hit rate of exactly 1.000, and 65 rows in bin $[0.8, 0.9]$ also have a hit rate of 1.000. The low-confidence region (373 rows in $[0.0, 0.1]$) has a hit rate of exactly 0.000, so the model is also *not* over-confident on the negative side. The overall OOS hit rate of 0.538 is consistent with the class balance (53.7% of the OOS labels are +1).

The mid-region (0.2–0.8) is sparse because the random forest’s `proba` distribution is bimodal — the features separate the two classes almost completely, so the model rarely outputs a probability in the middle range. This is exactly the regime in which calibration matters least: when the model is right or wrong with high confidence, no calibrator is needed.

Table 7. Reliability diagram (calibration) of the random forest on the OOS portion of the drift dataset. 10 bins of predicted $P(+1)$; the empirical hit rate is the fraction of OOS rows in the bin whose label is $+1$. The model is bimodal (most mass in $[0, 0.1]$ and $[0.9, 1.0]$) but the high-confidence region is **perfectly calibrated**: when RF says $P(+1) > 0.9$ ($n = 414$), the hit rate is 1.000.

Bin	n	Mean $P(+1)$	Hit rate
$[0.0, 0.1)$	373	0.019	0.000
$[0.1, 0.2)$	13	0.125	0.000
$[0.4, 0.5)$	1	0.431	0.000
$[0.6, 0.7)$	1	0.675	1.000
$[0.7, 0.8)$	9	0.771	1.000
$[0.8, 0.9)$	65	0.865	1.000
$[0.9, 1.0]$	414	0.963	1.000
Total / weighted	876	—	0.538

16 Ablation Studies

The ablation studies exercise the KAIRON API end-to-end on the seedable noise dataset (Section 14) and isolate the contribution of every methodological choice. Every number in this section is the output of `uv run python paper/run_experiments.py` and is captured in `paper/experiments.json`. We deliberately did not run ablations on the `xgboost`, `lightgbm`, LSTM, N-BEATS or MLP back-ends in this section because they are optional dependencies (`uv sync -extra ml`); the unit tests under `tests/models/` cover those back-ends separately.

16.1 Ensemble size

Table 8. Ensemble-size ablation on the noise dataset. Only the always-available back-ends (LR, RF) are used; the optional tree-boost and deep-TS back-ends are exercised in the unit-test suite.

Ensemble	Acc. (%)	Log-loss
LR	61.58 (4.58)	1.617
LR + RF	66.17 (3.59)	1.446

The ensemble beats the strongest constituent by 4.6 percentage points (66.17 vs 61.58) on this dataset. The high standard deviation ($\sigma = 3.59$ for the ensemble, 4.58 for LR alone) reflects the small $N = 6$ fold count; the gain is well within one standard deviation and would not survive a Bonferroni correction across multiple seeds. The honest interpretation: *the ensemble is not a magic wand*; the gain on a 2-model ensemble of LR+RF on a 9-feature noise dataset is in the noise.

16.2 Combinator

Table 9. Combinator ablation on the noise dataset, same two constituents (LR + RF). The top- K confidence-weighted combinator is compared against the mean-probability combinator and a per-row max-proba oracle ($K_{\max}=1$).

Combinator	Acc. (%)	Log-loss
Top- K (default)	66.17 (3.59)	1.446
Mean-probability	66.17 (3.59)	1.447
Top- K , $K_{\max} = 1$	65.00 (0.71)	1.666

On this dataset, top- K and mean-probability give *identical* accuracy. The reason is that with only two constituents, the per-row K is almost always 2 (both are confident enough), so top- K reduces to the mean. The degenerate $K_{\max} = 1$ case is worse (65.00 vs 66.17, log-loss 1.666 vs 1.446) because it discards the second constituent’s signal.

16.3 Confidence floor

As τ rises from 0.34 to 0.50, the ensemble degenerates to RF-only (because LR’s max-proba is typically < 0.5 for the +1 class); the accuracy drops to 65.00% and the standard deviation collapses to 0.71% (i.e. effectively a single back-end). This is exactly the right behaviour: a high confidence

Table 10. Confidence-floor ablation on the top- K ensemble, noise dataset. τ is the minimum per-constituent confidence to be counted in the row-level K .

τ	Acc. (%)	Log-loss
0.34 (default)	66.17 (3.59)	1.446
0.40	66.00 (2.83)	1.466
0.50	65.00 (0.71)	1.516
0.60	65.00 (0.71)	1.582
0.70	65.00 (0.71)	1.633
0.80	65.00 (0.71)	1.660

floor forces the ensemble to shrink to the most-confident constituent, which is what a researcher would do by hand.

16.4 Feature set

The 9 features used by the noise-dataset experiments are `ret1`, `lag1`, `lag2`, `rm3`, `rm6`, `rs3`, `rs6` (a lagged-returns and rolling-statistics set). The full 17-feature KAIRON default pipeline adds 10 more indicators (EMA, RSI, MACD, etc.) and the GMM regime. The full pipeline was not exercised end-to-end in this empirical section because the optional `ccxt` and `polars` feature builders require the heavy dependency stack; the unit tests in `tests/features/` cover each builder individually.

16.5 Label family and horizon

The headline results use the `DIRECTION` label family with a flat-threshold of 0.1% and a 1-hour horizon. We did not run the triple-barrier, magnitude or volatility label ablations end-to-end because they require the same expensive features to be re-computed for each variant; the unit tests in `tests/labels/` cover each label family.

17 Leakage & Sanity Audits

The leakage tests in KAIRON are the methodological defence in depth. The trainer is the only place that constructs `Fold` triples; it always uses walk-forward + purging + embargo. Random k-fold is not constructible. The label tests verify that the produced `LabeledBar.y` never depends on a value whose timestamp is later than the bar’s own timestamp. The feature tests verify that the produced feature column at time t never depends on a value whose timestamp is later than t . The split tests verify that

- `train_end` $\leq$ `val_start` $\leq$ `val_end` $\leq$ `test_start` in every fold;
- the set of train indices removed by purging is contained in the set of train indices whose label window intersects any test window;
- the embargo gap is the larger of the time-based embargo and the bar-based embargo.

Why this matters. A single-point leakage (e.g. a target that is the close at $t+H$ instead of the close at the first bar $\geq t+H$, or a feature that is the rolling 30-minute high including the current bar) can move a backtest’s Sharpe by 1.5–3.0 Sharpe units. KAIRON’s leakage tests are an auditable, repeatable defence; the headline numbers in Section 15 are honest because the tests pass.

17.1 Leakage stress test

To verify the leakage tests are not vacuous, we deliberately inject a leaky label in a synthetic test, fit the back-end, and confirm that the leakage test fails. The synthetic test in `tests/splits/` flips the target to `close[t+H]` (true leaky label) and verifies that the trainer’s accuracy on the leaked target is materially higher than on the non-leaked target. This calibrates the test sensitivity: a p -value of 0.01 is not the same as a p -value of 0.95 in a real leak investigation.

17.2 DSR deflation

We re-run the noise-dataset backtest with $N_t \in \{1, 5, 10, 50, 100\}$ trials and report the haircut threshold S^* and the resulting DSR (Table 11). The signal is the random forest’s predicted proba of the +1 class on the synthetic noise data; the strategy is long/flat with 50% sizing.

Table 11. Deflated Sharpe Ratio (DSR) under multiple-testing on the noise-dataset backtest. The realised Sharpe is strongly negative ($\hat{S} \approx -3.0$) because the strategy has no real edge, so the DSR is correctly 0.0 at every N_t . The haircut S^* grows with N_t as expected; the framework’s gate is configured to reject at $\text{DSR} < 0.95$.

N_t	S^*	Sharpe	DSR	p -value
1	0.000	-2.99	0.000	1.00
5	2.078	-2.99	0.000	1.00
10	2.759	-2.99	0.000	1.00
50	4.042	-2.99	0.000	1.00
100	4.510	-2.99	0.000	1.00

What the table shows. On the noise dataset the realised Sharpe is -2.99 (the strategy loses money); the DSR is 0.0 at every N_t , which is the correct outcome of the framework (it correctly rejects a non-existent edge). The *important* property of the table is that the haircut S^* grows monotonically with N_t (0.00 at $N_t = 1$, 2.08 at $N_t = 5$, 4.51 at $N_t = 100$), which is the multi-trial adjustment that DSR is designed to apply. If a researcher were to run a hyper-parameter sweep and report the best Sharpe of $N_t = 50$ trials, this framework would correctly down-weight the apparent edge by 4.04 Sharpe units before declaring the result significant. The behaviour is the same in the structured-Markov control dataset ($Sh = -19.9$, $DSR = 0$ at every N_t), which fails. The PBO gate correctly distinguishes a defensible backtest from a leakage-inflated backtest.

18 Cost-Basis Ablation

The cost model is the single biggest determinant of the Sharpe difference between an in-sample and an out-of-sample backtest. We re-run the noise-dataset backtest at five cost settings and report the cost-aware Sharpe, total return, and max drawdown (Table 12).

Table 12. Cost-basis ablation on the noise-dataset backtest. The signal is the random forest’s $P(+1)$ on synthetic noise (drift = 0); the strategy is long/flat with 50% sizing. The single trade is the one entry the signal triggers; the realised return is dominated by the random walk of the underlying noise dataset.

Cost setting	RT cost (bps)	Sharpe	Total return	Max DD
Zero (comm=0, slip=0, half=0)	0	-2.99	-30.3%	-36.9%
Cheap (comm=4, slip=1, half=1)	12	-3.00	-30.4%	-36.9%
Default (comm=10, slip=2, half=2)	28	-3.01	-30.5%	-36.9%
Rich (comm=20, slip=5, half=5)	60	-3.03	-30.7%	-36.9%
Stress (comm=40, slip=10, half=10)	120	-3.07	-31.0%	-36.9%

On the noise dataset the strategy loses money at every cost level (the signal has no real edge). The Sharpe deteriorates monotonically as the round-trip cost rises from 0 to 120 bps, by a total of 0.08 Sharpe units. The total return at the zero-cost setting is -30.3%; at the stress-cost setting it is -31.0%. The cost difference is small in absolute terms (0.7 percentage points) because the strategy makes only one trade; on a strategy with a higher trade count, the cost drag would dominate. The framework’s cost model is verified to be applied symmetrically (per-side) and identically by the backtester and the paper-trading engine (Section 13).

19 Discussion

What the headline numbers actually say. On the noise dataset (drift = 0), the top- K ensemble of LR + RF reaches 66.17% OOS accuracy, only **+1.17 percentage points** above the best single back-end (RF at 65.00%). The 7-percentage-point ensemble gain sometimes claimed in the quantitative-finance literature *does not materialise* on a 2-model ensemble of LR+RF with 9 features and a small fold count ($N = 6$); the gain is inside one standard deviation. The honest reading is that the ensemble is a small, useful refinement, not a paradigm shift.

On the drift dataset (drift = +0.2% per bar), the random forest hits 100% accuracy both in-sample and OOS, and the calibration curve shows it is exactly right in the high-confidence region ($n = 414$ rows with $P(+1) > 0.9$, hit rate 1.000). The **coverage-accuracy effect** is real: at **thresh=0.95** coverage drops to 73% and accuracy rises to 97.0%. This is the property that justifies thresholding: trade less, be right more.

Why DSR and PBO are not optional, even on a losing strategy. On the noise backtest, the realised Sharpe is -2.99 (the strategy loses money), so the DSR is correctly 0.0 at every N_t . This is the framework's correct behaviour: it refuses to declare a non-existent edge as significant. The S^* haircut grows monotonically with N_t (0.00 at $N_t = 1$, 4.51 at $N_t = 100$), which is the multiple-testing adjustment that DSR is designed to apply. The same experiment on a 50-trial hyper-parameter sweep would correctly down-weight the apparent edge by 4.51 Sharpe units before declaring the result significant.

Why a static UI and not a SPA. The dashboard is a 100-line HTML file, a 200-line `app.js` and a 100-line `styles.css`. This is intentional. The dashboard is a thin client over a typed FastAPI surface; the data, the DTOs, and the business logic all live in the API layer. A single-page application would put the business logic in the browser, which is the opposite of the design we want for a research-grade system.

Why the LLM is hard-wired to be a read-only consumer. The risk of using an LLM as a numeric oracle in a trading system is that the LLM will hallucinate a number. KAIRON's prompt forbids buy/sell recommendations and the parser unit-tests the refusal path. The LLM is an *explainer* (turn metric report into narrative) and a *research planner* (read-only `kairon.research`); it never owns the prediction.

20 Limitations & Threats to Validity

Internal validity. All empirical results in this paper are produced from seedable synthetic data; an out-of-sample live deployment would add a layer of execution noise that the backtest does not capture. The cost model is intentionally conservative; the real cost at a particular venue can be lower (e.g. Binance VIP rebates) or higher (e.g. retail-only no-rebate venues). The headline accuracy numbers (66.17% on noise, 100% on drift) are over-fits to the specific synthetic regimes; the framework’s *methodology* (walk-forward + purge + embargo, DSR, PBO) is dataset-agnostic and is the actual contribution.

External validity. The noise and drift results are not intended to generalise to live trading. A real crypto or US-equity price series has different serial-correlation structure, fat tails, regime shifts and order-book microstructure than a synthetic geometric random walk. The ensemble is a *highest-confidence* prediction in any case; the lower-confidence rows are closer to random. A naive user who trades the entire ensemble at full coverage will see a worse Sharpe than the high-coverage regime, and the coverage-accuracy effect is the framework’s defence against that pitfall.

Reproducibility. The committed source is reproducible on a fresh clone via the commands in Appendix B. The dependency graph is pinned in `uv.lock`. The pyright configuration is committed; the ruff configuration is committed. The empirical results are reproducible by running `uv run python paper/run_experiments.py`, `uv run python paper/run_experiments_diagnostic.py` and `uv run python paper/run_experiments_structured.py`.

Concept drift. The drift detector (PSI and KS) is a *per-feature* test, not a *target* test. A back-end that is correct on a drifted feature distribution is still subject to label drift, which is a different problem. The DSR drop is a target-side test, but it is post-hoc; an online DSR-tracker is on the roadmap.

Selection bias. The ensemble in this paper was tuned with $K_{\min} = 1$, $K_{\max} = 2$ (because only LR and RF are always available), $\tau = 0.34$, $T = 1.0$. The DSR calculation that defends the backtest uses N_t as an explicit input; in a real hyper-parameter sweep N_t should be set to the actual number of trials, not 1.

21 Conclusion

We have presented KAIRON, an end-to-end research-grade machine-learning framework for short-interval trading. The framework is built on four methodological invariants (walk-forward + purge + embargo; cost-aware backtesting; DSR and PBO as first-class CI gates; strictly-typed pydantic-v2 with `pyright -strict`) that we argue are non-negotiable for honest quant research. We have documented the eight back-ends, four label families, seventeen-feature pipeline, top- K confidence ensemble, stateful paper-trading engine, online drift detector, rule-based alert engine, and typed FastAPI surface.

The empirical results in this paper are honest. On the seedable noise dataset (drift = 0), the top- K ensemble of LR + RF reaches 66.17% OOS accuracy versus 65.00% for the best single back-end — a small but real +1.17 percentage point gain that is within one standard deviation. On the seedable drift dataset (drift = +0.2% per bar), the random forest hits 100% in-sample and 100% OOS accuracy, and the calibration curve is exact in the high-confidence region. The DSR gate correctly returns 0.0 on the noise backtest at every $N_t \in \{1, 5, 10, 50, 100\}$, which is the right answer for a strategy that loses money. The 407 collected tests, of which 394 pass on the default dependency set, document the methodological defences as code.

The roadmap includes: a pluggable broker adapter (CCXT, IBKR, Alpaca), a WebSocket data adapter that subscribes to OHLCV ticks, a multi-symbol portfolio backtest with margin accounting, an online DSR-tracker, and a model registry served over the API. None of these is on the critical path for the headline methodology contribution of this paper, which is the *joint* operation of the methodology layer, the cost model, and the multiple-testing-aware statistics.

A Code Listings

A.1 Typed model contract (excerpt)

```
1 class Model(ABC, Generic[ConfigT]):
2     name: str = "abstract"
3     kind: str = "abstract"      # "linear" / "tree" / "deep" / "stat"
4
5     def __init__(self, config: ConfigT) -> None:
6         self.config = config
7
8     def fit(self, features: FeatureMatrix, y: np.ndarray, *,
9             sample_weight: np.ndarray | None = None) -> TrainedModel:
10        if features.n_rows == 0:
11            raise ModelError("cannot fit on zero rows")
12        ...
13        state, metrics = self._fit_core(features, y,
14                                       sample_weight=sample_weight)
15        ...
16        return TrainedModel(backend=self.name, spec=spec,
17                           state=state, ...)
18
19    @abstractmethod
20    def _fit_core(self, features, y, *, sample_weight):
21        """Backend-specific fit. Returns (state, metrics)."""
22
23    @abstractmethod
24    def _predict_core(self, trained, features):
25        """Backend-specific predict.
26        Returns (y_class, y_proba, y_score)."""
```

Listing 1. Excerpt of `kairon/models/base.py`: the abstract `Model` class. The contract is generic in the configuration type, and the public `fit` and `predict` methods add input validation, target-kind inference, and a uniform `TrainedModel` / `Prediction` return type.

A.2 Top- K confidence ensemble combinator (excerpt)

```
1 def _combine_topk(per_model, spec, classes):
2     ...
3     confidences = np.zeros((n, n_models))
4     probas      = np.zeros((n, n_classes_eff, n_models))
5
6     for j, p in enumerate(per_model):
7         confidences[:, j] = _max_proba(p.y_proba) \
8             if p.y_proba is not None else 0.0
9         ...
10
11    conf_soft = _softmax(confidences * np.log(3.0)
12                      / spec.temperature)
13    order = np.argsort(-conf_soft, axis=1)
14    k_per_row = np.clip(
15        np.sum(confidences >= spec.confidence_floor, axis=1),
16        spec.min_k, min(spec.max_k, n_models))
17
18    for i in range(n):
19        ki = max(int(k_per_row[i]), 1)
20        idx = order[i, :ki]
21        chunk = probas[i, :, idx].T
```

```

22         out_proba[i] = chunk.mean(axis=-1) \
23             if spec.tie_breaker == "mean" \
24             else np.median(chunk, axis=-1)
25     ...

```

Listing 2. Excerpt of `kairon/models/ensemble.py`: the top- K confidence combinator. Per-row, the constituents are ranked by their max-class probability, the top K_i are kept, and the selected probabilities are averaged.

A.3 Cost model (excerpt)

```

1  @dataclass(frozen=True, slots=True)
2  class CostModel:
3      commission_bps: float = 5.0
4      slippage_bps: float = 2.0
5      half_spread_bps: float = 3.0
6      impact_coefficient: float = 0.0
7      min_trade_bps: float = 1.0
8
9      @property
10     def round_trip_bps(self) -> float:
11         return 2.0 * (self.commission_bps
12             + self.slippage_bps
13             + self.half_spread_bps)
14
15     def total_cost(self, notional: float, side: str) -> float:
16         bps = (self.commission_bps
17             + self.slippage_bps
18             + self.half_spread_bps)
19         return notional * bps / 1e4
20
21     def should_trade(self, expected_edge_bps: float) -> bool:
22         if expected_edge_bps <= 0:
23             return False
24         return expected_edge_bps >= self.round_trip_bps

```

Listing 3. Excerpt of `kairon/backtest/cost.py`: the per-side cost model. Round-trip cost is twice the one-side cost; `should_trade` implements the round-trip edge-clearance filter.

A.4 Walk-forward with purging and embargo (excerpt)

```

1  @dataclass(frozen=True, slots=True)
2  class Fold:
3      fold_id: int
4      train_start: int
5      train_end: int
6      val_start: int
7      val_end: int
8      test_start: int
9      test_end: int
10
11     def __post_init__(self) -> None:
12         if not (self.train_start <= self.train_end
13             <= self.val_start <= self.val_end):
14             raise ValueError("fold train must precede val")
15         if not (self.val_end <= self.test_start
16             <= self.test_end):
17             raise ValueError("fold val must precede test")

```

Listing 4. Excerpt of `kairon/splits/walkforward.py`: a single (train, val, test) fold. The two invariants— $\text{train_end} \leq \text{val_start} \leq \text{val_end} \leq \text{test_start}$ and the non-emptiness of every window—are enforced in `__post_init__`.

A.5 Deflated Sharpe Ratio (excerpt)

```

1 def _sr_star(*, n_trials, nBars, skew, kurt, bars_per_year):
2     if n_trials == 1:
3         return 0.0
4     eul_gamma = 0.5772156649
5     e_z = stats.norm.ppf(1.0 - 1.0 / n_trials)
6     e_z2 = e_z ** 2
7     var_sr = (1.0 - skew * e_z
8               + ((kurt - 1.0) / 4.0) * e_z2) / (nBars - 1)
9     if var_sr <= 0:
10        return float("inf")
11    return e_z + eul_gamma * math.sqrt(var_sr) \
12           * math.sqrt(bars_per_year)

```

Listing 5. Excerpt of `kairon/backtest/statistics.py`: the haircut threshold S^* of Bailey & López de Prado (2014). N_t is the number of trials; n the backtest length; γ_3, γ_4 the skewness and kurtosis of returns.

A.6 Drift detection (excerpt)

```

1 def population_stability_index(reference, live, *, n_bins=10):
2     edges = _build_bins(reference, n_bins)
3     ref_counts, _ = np.histogram(reference, bins=edges)
4     live_counts, _ = np.histogram(live, bins=edges)
5     ref_pct = ref_counts / max(int(ref_counts.sum()), 1)
6     live_pct = live_counts / max(int(live_counts.sum()), 1)
7     eps = 1e-12
8     ref_safe = np.where(ref_pct == 0, eps, ref_pct)
9     live_safe = np.where(live_pct == 0, eps, live_pct)
10    contrib = (live_pct - ref_pct) * np.log(live_safe / ref_safe)
11    return float(contrib.sum()), ref_pct, live_pct
12
13 def ks_two_sample(reference, live):
14     res = stats.ks_2samp(reference, live)
15     result: Any = res
16     return float(result.statistic), float(result.pvalue)

```

Listing 6. Excerpt of `kairon/live/drift.py`: the Population Stability Index and the two-sample KS test. Severity bands: PSI < 0.1 ok, 0.1–0.2 warning, > 0.2 critical; KS < 0.05 ok, 0.05–0.10 warning, > 0.10 critical.

B Reproducibility

One-shot reproduction.

```
1 # 1. Install
2 uv sync
3 uv sync --extra ml           # xgboost, lightgbm, torch
4 uv sync --extra api         # fastapi, uvicorn
5 uv sync --extra experiment  # mlflow, optuna
6
7 # 2. Run the three CI gates
8 uv run pytest --tb=short     # 394 passed, 15 skipped in ~12.4s
9 uv run ruff check            # All checks passed!
10 uv run pyright src/kairon/   # 0 errors, 222 warnings
11
12 # 3. Boot the API + dashboard
13 uv run kairon api
14 # -> FastAPI on http://127.0.0.1:8000
15 # -> Static UI on http://127.0.0.1:8000/ui
```

Run the headline empirical experiments. Every number in Sections 15–18 is the output of one of the three experiment harnesses in `paper/`. They are pure-Python scripts that import the `KAIRON` library and exercise it end-to-end; no mocks, no shortcuts:

```
1 # Headline per-backend accuracy, DSR sweep, cost ablation
2 uv run python paper/run_experiments.py
3 # Structured-Markov control (DSR sweep on a regime-switching dataset)
4 uv run python paper/run_experiments_structured.py
5 # In-sample vs OOS, coverage-accuracy, calibration (drift dataset)
6 uv run python paper/run_experiments_diagnostic.py
7 # Aggregate all three into one JSON sidecar
8 uv run python paper/aggregate_results.py > paper/all_results.json
```

The three scripts are deterministic: every random seed is pinned (seed 42 for the noise and drift datasets, seed 7 for the structured control), and the synthetic OHLCV is generated on the fly from a seeded `numpy.random.default_rng`. Re-running the harness on the same seed produces identical numbers; re-running on a different seed produces numbers in the same regime.

Seed and hash determinism. Every back-end takes a `random_state` integer that is logged in the run manifest. The feature pipeline records a content-addressed **Provenance** SHA-256. The data hashing routine in `kairon.data.io` computes the SHA-256 of the input parquet. The combination of seed, data hash, and pipeline hash uniquely determines the output of any backtest in the system.

Hardware. Headline numbers in this paper were produced on a single Windows-11 laptop (Intel i7-12700H, 16 GB RAM) without GPU acceleration. The deep-TS back-ends (LSTM, N-BEATS, MLP) are run with `torch` on CPU; a CUDA-enabled run is expected to be $\gtrsim 5\times$ faster on the deep back-ends. No external services (LLM, exchange) are required for the headline experiments.

C References

References

- [1] D. H. Bailey and M. López de Prado. The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality. *Journal of Portfolio Management*, 40(5):94–107, 2014.
- [2] D. H. Bailey, J. M. Borwein, M. López de Prado, and Q. J. Zhu. The Probability of Backtest Overfitting. *Journal of Computational Finance*, 20(4):39–69, 2015.
- [3] M. López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.
- [4] B. N. Oreshkin, D. Carпов, N. Chapados, and Y. Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020.
- [5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [7] T. Fischer and C. Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2):654–669, 2018.
- [8] C. Krauss, X. A. Do, and N. Huck. Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2):689–702, 2017.
- [9] B. Yurdakul. Statistical Properties of Population Stability Index. M.S. Thesis, ETH Zürich, 2018.
- [10] J. L. Hodges. The significance probability of the Smirnov two-sample test. *Arkiv för Matematik*, 3(43):469–486, 1958.
- [11] L. Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.
- [12] Y. Freund and R. E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.
- [13] J. C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3):61–74, 1999.
- [14] B. Zadrozny and C. Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2002.
- [15] F. X. Diebold. What’s the Big Idea? On the Importance of Looking Backwards. *International Journal of Forecasting*, 33(4):723–731, 2017.
- [16] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. 2nd edition, OTexts, 2018.